```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity shiftreg is

  Port (
    clk : in STD_LOGIC;
    rst : in STD_LOGIC;
    load : in STD_LOGIC;
    si : in STD_LOGIC;

       pi : in STD_LOGIC_VECTOR(3 downto 0);
       po : out STD_LOGIC_VECTOR(3 downto 0);

       mode : in STD_LOGIC_VECTOR(1 downto 0);
       so : out STD_LOGIC
  );

end shiftreg;


architecture Behavioral of shiftreg is

  signal clkout : STD_LOGIC;
  signal temp : STD_LOGIC_VECTOR(3 downto 0);
```

```
 signal counter : integer range 0 to 49999999;

begin


 -- CLOCK DIVIDER
 process(clk, rst)
 begin

    if(rst = '1') then

      clkout <= '0';
      counter <= 0;

    elsif(clk'event and clk = '1') then
  if(counter = 49999999) then

    clkout <= not clkout;
    counter <= 0;

  else

    counter <= counter + 1;

  end if;

 end if;
```

```
end process;



-- SHIFT REGISTER
process(clkout, rst)
begin

 if(rst = '1') then

   so <= '0';
   po <= "0000";
   temp <= "0000";

 elsif(clkout'event and clkout = '1') then

   case mode is


     -- PIPO
     when "00" =>

         po <= pi;
         so <= '0';



     -- SISO
```

```
   when "01" =>

       temp(0) <= si;
       temp(1) <= temp(0);
       temp(2) <= temp(1);
 temp(3) <= temp(2);

 so <= temp(3);

 po <= "0000";



-- PISO
when "10" =>

 if(load = '1') then

   temp <= pi;

 else

   temp(2) <= temp(3);
   temp(1) <= temp(2);
   temp(0) <= temp(1);
   temp(3) <= '0';

   so <= temp(0);
```

```
    end if;



  -- SIPO
  when others =>

    if(load = '0') then

      temp(0) <= si;
      temp(1) <= temp(0);
      temp(2) <= temp(1);
      temp(3) <= temp(2);

    else

      po <= temp;

    end if;



end case;
    end if;

  end process;

end Behavioral;
```